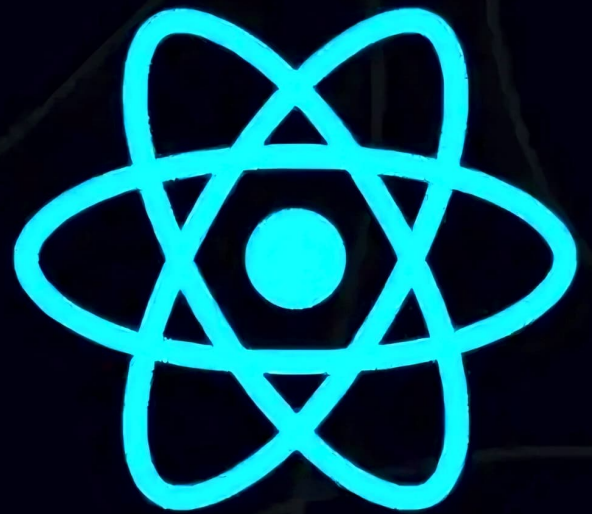


Hooks in React



What are Hooks ?

Hooks are special **functions** in React that allow you to use **state** and other React features in functional components.

They were introduced in **React 16.8** to let developers use stateful logic without writing **class components**.



Key Concepts

State Management: Hooks like `useState` allow you to add **state** to functional components.

Side Effects: Hooks like `useEffect` let you perform side effects in your components, such as **data fetching**, **subscriptions**, or manual **DOM manipulations**.

Reusability: Custom hooks enable the **reuse of stateful logic** across different components without changing their design.

Commonly Used Hooks

useState :

Allows you to add state to a functional component.

Syntax :

```
const [state, setState] = useState(initialValue);
```

useState :

Manages side effects such as data fetching, subscriptions, or manually changing the DOM.

Syntax :

```
useEffect(effectFunction, dependencies);
```

useContext :

Accesses React context without using **Context.Consumer**

Syntax :

```
const value = useContext(Context);
```

useReducer :

Manages complex state logic, similar to useState but with a reducer function.

Syntax :

```
const [state, dispatch] = useReducer(reducer, initialState);
```

useRef :

Provides a way to access and interact with DOM elements or persist values across renders without causing a re-render

Syntax :

```
const ref = useRef(initialValue);
```

Custom Hooks :

Custom hooks are functions that start with "**use**" and allow you to **reuse stateful logic**. They can call other hooks and perform reusable behavior.

Note: Hooks must be called at the **top level** of a component or custom hook, not inside loops, conditions, or nested functions.